SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks:25**

Q1. Transfer Learning
A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet
A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder
A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

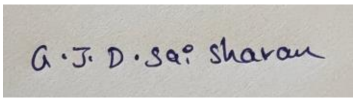
Q4. BPTT and LSTM
A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution
A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Code of Conduct:

I _G J D Sai Sharan/192425152_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understandin g; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Q1. Transfer Learning

Problem Understanding

The healthcare startup has only a limited number of labeled dermoscopic images for skin-cancer classification. Training a deep CNN from scratch would require a very large dataset and may cause overfitting. Therefore, **transfer learning** using a CNN pre-trained on a large image dataset is appropriate.

Proposed Model

I would use **EfficientNet-B0** pre-trained on ImageNet. EfficientNet provides strong image-classification performance with relatively few parameters, making it suitable for a healthcare application with limited training data and computational resources.

Architecture

Dermoscopic Image
↓
Image Preprocessing
(Resize + Normalize + Augmentation)
↓
Pre-trained EfficientNet-B0
↓
Frozen Convolutional Layers
↓
Fine-tuned Higher Layers
↓
Global Average Pooling
↓
Dropout
↓
Fully Connected Layer
↓
Softmax / Sigmoid
↓
Benign / Malignant

Transfer Learning Strategy

Initially, all convolutional layers of EfficientNet-B0 are **frozen**. The original ImageNet classification head is removed and replaced with a new classification layer suitable for skin-cancer classes.

For example:

- **Early layers:** Frozen because they learn general features such as edges, textures, and shapes.
- **Middle layers:** Initially frozen and later selectively fine-tuned.
- **Higher convolutional layers:** Fine-tuned because they contain more task-specific visual representations.
- **New classification head:** Fully trainable.

A suitable learning rate for fine-tuning would be smaller than that used for training the new classification layer, preventing the destruction of useful pre-trained features.

Data Preprocessing

Dermoscopic images can be resized to a common input size and normalized. Data augmentation such as:

- Rotation
- Horizontal/vertical flipping
- Cropping
- Scaling
- Brightness adjustment

can increase the effective training diversity and reduce overfitting.

Why Transfer Learning Improves Performance

Transfer learning allows the model to start with features learned from millions of images instead of random weights. The model therefore requires fewer labeled medical images and generally converges faster.

Fine-tuning the higher layers allows the network to adapt general visual features to skin-specific characteristics such as:

- Lesion shape
- Color variation
- Texture
- Border irregularity
- Pigmentation patterns

Class-weighted loss or focal loss can additionally help when malignant cases are much less frequent than benign cases.

Performance Evaluation

The model should be evaluated using:

- Accuracy
- Precision

- Recall/Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion matrix

For medical diagnosis, **sensitivity/recall is especially important**, because incorrectly classifying a malignant lesion as benign can have serious consequences.

Thus, EfficientNet-based transfer learning provides a practical solution that reduces training requirements, improves generalization, and achieves better classification performance than training a CNN from scratch on a small dataset.

Q2. DenseNet and PixelNet

Problem Understanding

The smart-city system must identify roads, pedestrians, vehicles, and traffic signs **at pixel level**. Therefore, ordinary image classification is insufficient. A semantic segmentation architecture is required.

A suitable solution combines **DenseNet** for feature extraction with a **PixelNet-style pixel-level prediction network** for dense segmentation.

Proposed Architecture

Road Scene Image
↓
Preprocessing
↓
DenseNet Backbone
↓
Dense Feature Maps
↓
Multi-level Feature Extraction
↓
PixelNet Pixel-wise Feature Representation
↓
Segmentation Head
↓
1 × 1 Convolution
↓
Softmax
↓
Pixel-level Class Map

↓
Road / Pedestrian / Vehicle / Sign / Background

Role of DenseNet

DenseNet connects each layer to subsequent layers within a dense block. Instead of learning completely new representations at every layer, later layers receive feature maps from earlier layers.

Conceptually:

x0 → Layer1 → Layer2 → Layer3
↘ ↘ ↘
Dense Connections

For a layer (l):

[

x_l = H_l([x_0,x_1,...,x_{l-1}])

]

where ([ ]) represents concatenation.

This provides strong **feature reuse** and improves gradient propagation.

Role of PixelNet

PixelNet performs pixel-level prediction by learning representations for individual image pixels. Instead of predicting one label for the complete image, the network generates a classification prediction for every pixel.

For example:

Input Image

| Road | Car | Sign |
| Road | Car | Ped |

Output Segmentation

| R | V | S |
| R | V | P |

where R = road, V = vehicle, S = sign, and P = pedestrian.

Improving Segmentation Accuracy

DenseNet provides rich hierarchical features:

- Low-level layers → edges and textures
- Middle layers → object parts
- High-level layers → semantic information

PixelNet then converts these features into pixel-level predictions.

Skip connections or multi-scale feature fusion can be added to preserve spatial details that may otherwise be lost during downsampling.

Computational Efficiency

DenseNet encourages feature reuse, reducing unnecessary duplicate feature learning. It can achieve strong representation quality with fewer parameters than many traditional CNN architectures.

PixelNet can further improve efficiency by sampling representative pixels during training rather than processing every possible pixel in an expensive manner.

Loss Function

A pixel-wise cross-entropy loss can be used:

$$L=-\sum_{i=1}^{N}\sum_{c=1}^{C}y_{ic}\log(p_{ic})$$

For highly imbalanced classes such as traffic signs, class-weighted loss can be used.

Evaluation

The system can be evaluated using:

- Pixel accuracy
- Mean Intersection over Union (mIoU)
- Precision
- Recall
- F1-score
- Class-wise IoU

Therefore, DenseNet provides efficient feature extraction and feature reuse, while PixelNet provides dense pixel-level classification. Their combination produces an efficient and accurate road-scene segmentation system.

Q3. Bidirectional RNN and Encoder–Decoder

Problem Understanding

The multinational company requires a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Translation requires understanding the complete source sequence and generating a target sequence of potentially different length.

An **Encoder–Decoder sequence-to-sequence architecture with Bidirectional RNNs** is suitable.

Proposed Architecture

```
Source Sentence
"I need help with my account"
           ↓
     Tokenization
           ↓
   Bidirectional RNN Encoder
      ↙           ↘
Forward RNN      Backward RNN
      ↘           ↙
   Context Representation
           ↓
      Decoder RNN
           ↓
    Attention Mechanism
           ↓
     Target Language
           ↓
"I need assistance..."
```

Bidirectional Encoder

A normal RNN processes a sequence in one direction:

[

$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow ... \rightarrow x_T$

]

A Bidirectional RNN processes it in both directions:

[

$$\overrightarrow{h_t}=RNN(x_t,\overrightarrow{h_{t-1}})$$

]

[

$$\overleftarrow{h_t}=RNN(x_t,\overleftarrow{h_{t+1}})$$

]

The final representation is:

[

$$h_t=[\overrightarrow{h_t};\overleftarrow{h_t}]$$

]

Thus, each word representation contains information from both its previous and following context.

Encoder

The encoder converts the input sentence into contextual representations.

For example:

Customer:
"Can you cancel my flight tomorrow?"
↓
Bi-RNN Encoder
↓
Contextual representations

The encoder captures relationships between words and their surrounding context.

Decoder

The decoder generates the translated sentence one token at a time:

[

$$P(y|x)=\prod_{t=1}^{T}P(y_t|y_{<t},x)$$

]

At every step, the decoder uses its previous output, hidden state, and encoder information.

Attention Mechanism

Attention allows the decoder to focus on the most relevant source words when generating each target word.

This is particularly useful for long sentences because the entire source sentence does not have to be compressed into a single fixed-size vector.

Why Bidirectional Processing Improves Translation

Consider:

"The customer did not cancel the order because it was delayed."

Understanding the meaning of "it" requires surrounding context. A Bidirectional RNN can use both preceding and following information.

Advantages include:

- Better contextual understanding
- Improved handling of long sentences
- Better word disambiguation
- Improved translation accuracy
- Better handling of multilingual grammatical structures

For production deployment, the system can use language-specific tokenizers or multilingual subword tokenization and an LSTM/GRU-based decoder.

Q4. BPTT and LSTM

Problem Understanding

Stock prices depend on historical information such as previous prices, trading volume, market indicators, and temporal trends. Therefore, a recurrent model is suitable because the current prediction can depend on previous observations.

However, traditional RNNs can suffer from vanishing and exploding gradients when learning long-term dependencies. An **LSTM** addresses this problem through gated memory.

Proposed System

Historical Market Data
↓
Data Cleaning
↓
Normalization
↓
Sliding Time Window
↓
LSTM Layer
↓
Dropout
↓
Dense Layer
↓
Predicted Stock Price

For example, a 30-day input window can be used to predict the next day's closing price.

Input features may include:

- Open price
- High price
- Low price
- Closing price
- Trading volume
- Technical indicators

LSTM Architecture

An LSTM contains a cell state and three major gates.

Forget Gate

[

$$f_t=\sigma(W_f[h_{t-1},x_t]+b_f)$$

]

Determines what previous information should be discarded.

Input Gate

[

$$i_t=\sigma(W_i[h_{t-1},x_t]+b_i)$$

Determines what new information should be stored.

Candidate Memory

$$\tilde{C_t}=\tanh(W_C[h_{t-1},x_t]+b_C)$$

Cell State

$$C_t=f_tC_{t-1}+i_t\tilde{C_t}$$

Output Gate

$$o_t=\sigma(W_o[h_{t-1},x_t]+b_o)$$

The hidden state is:

$$h_t=o_t\tanh(C_t)$$

Backpropagation Through Time

BPTT trains recurrent networks by unfolding the LSTM across the time sequence.

```
t1     t2     t3     t4
↓      ↓      ↓      ↓
LSTM → LSTM → LSTM → LSTM
↓      ↓      ↓      ↓
Prediction
     ↓
Loss
```

↓
Backward Through Time
↓
Update Weights

The total loss can be calculated over multiple time steps:

[

$$L=\sum_{t=1}^{T}L_t$$

]

Gradients are propagated backward through the unfolded network. The optimizer then updates the network parameters.

Why LSTM is Better Than Traditional RNN

Traditional RNNs repeatedly multiply gradients through time. This can cause gradients to become extremely small or extremely large.

LSTM's memory cell and gates provide a controlled path for information and gradients, allowing the network to preserve useful long-term dependencies.

This makes LSTM more suitable for financial time-series forecasting where information from earlier periods may influence later predictions.

Evaluation

The model can be evaluated using:

[

$$MAE=\frac{1}{N}\sum |y_i-\hat{y_i}|$$

]

and:

- MSE
- RMSE
- MAE
- (R^2)
- Directional accuracy

A chronological train-validation-test split should be used instead of randomly shuffling time-series data, thereby avoiding information leakage.

Q5. Integrated Deep Learning Solution

Problem Understanding

The media company wants to automatically generate captions from videos. This requires understanding both:

1. **Visual information** contained in video frames.
2. **Language information** required to generate grammatically meaningful captions.

Therefore, a combined **Transfer Learning + LSTM Encoder–Decoder** architecture is appropriate.

Proposed Architecture

```
        VIDEO
          ↓
     Frame Extraction
          ↓
   ┌──────────────────────────┐
   │ Pre-trained CNN   │
   │ Transfer Learning │
   └──────────────────────────┘
          ↓
   Visual Feature Vectors
          ↓
     LSTM Encoder
          ↓
   Context Representation
          ↓
     LSTM Decoder
          ↓
  Word Embedding + Softmax
          ↓
  "A man is playing football"
```

Step 1: Frame Extraction

The video is divided into frames at regular intervals.

For example:

Video
↓

Frame 1 → Frame 2 → Frame 3 → ... → Frame N

Sampling frames rather than processing every video frame reduces computational cost.

Step 2: Transfer Learning for Visual Features

A pre-trained CNN such as **ResNet50, EfficientNet, or InceptionV3** is used as the visual feature extractor.

The final classification layer is removed.

Video Frame
↓
Pre-trained CNN
↓
Convolutional Features
↓
Feature Vector

Initially, the CNN layers can be frozen because the model already contains useful visual features learned from a large dataset. Higher layers can later be fine-tuned using video-caption data.

Step 3: LSTM Encoder

The sequence of visual feature vectors is supplied to an LSTM encoder.

[

v_1,v_2,...,v_T

\rightarrow

h_1,h_2,...,h_T

]

The LSTM learns temporal relationships between frames.

For example:

Frame 1: Person
Frame 2: Person + Ball
Frame 3: Person kicking Ball
Frame 4: Ball moving

The LSTM can learn that these frames represent an event such as a person kicking a ball.

Step 4: LSTM Decoder

The decoder generates the caption sequentially.

<START>
↓
A
↓
man
↓
is
↓
playing
↓
football
↓
<END>

At every time step, the decoder predicts the probability of the next word.

[

$P(y_t|y_{<t},V)$

]

where (V) represents the extracted visual information.

Attention Mechanism

An attention mechanism can be added so that the decoder focuses on relevant video features while generating each word.

For example:

"man" → focus on person frames
"playing" → focus on action frames
"football" → focus on ball/football frames

This improves the relevance and accuracy of generated captions.

Training

The CNN feature extractor and LSTM caption generator can initially be trained separately. After the decoder becomes stable, selected CNN layers can be fine-tuned jointly with the captioning network.

Teacher forcing can be used during training, where the actual previous word is supplied to the decoder.

The model minimizes categorical cross-entropy:

$$L=-\sum_t\log P(y_t|y_{<t},V)$$

Evaluation

Caption quality can be evaluated using:

- BLEU
- ROUGE-L
- METEOR
- CIDEr
- Human evaluation

Advantages of the Integrated Solution

Transfer Learning provides powerful visual representations without requiring the company to train a CNN from scratch.

LSTM Encoder captures temporal information across video frames.

LSTM Decoder converts visual representations into natural-language captions.

Attention helps the decoder focus on relevant visual information.

Therefore, the complete system combines visual understanding, temporal modeling, and language generation to produce accurate and meaningful captions for accessibility.